

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

02 - Análise de Algoritmos e Conectividade Dinâmica

Prof. Alex Kutzke

Agosto de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Contar operações de um trecho de código em função de n e identificar o termo dominante;
2. Classificar uma função de custo com O , Ω e Θ , informando de qual caso se trata;
3. Estimar a ordem de crescimento de um programa por medição, com o teste de duplicação;
4. Explicar as três implementações de Union-Find, o custo de Find e Union em cada uma, e escolher qual usar para um tamanho de entrada dado.

Três programas, a mesma resposta

Na Aula 1, o desafio foi implementar Find e Union. Existe mais de uma forma de fazer isso.

```
cd 02/codes/unionfind
go build -o quick_find quick_find.go
go build -o quick_union quick_union.go
go build -o weighted weighted_quick_union.go

./weighted < largeUF.txt > /dev/null
```

Agora o mesmo arquivo, com `quick_find`. Quanto tempo leva?

800 vezes mais lento

weighted: **cerca de 1 segundo**

quick_find: **quase 14 minutos**

Mesma entrada, mesma máquina, mesmo compilador, menos de 140 linhas cada. A diferença vem só da estrutura de dados escolhida.

Medir essa diferença é a primeira metade da aula. Explicar de onde ela vem é a segunda.

Como decidir sem esperar 14 minutos?

Análise empírica

Implementar, rodar, cronometrar. Indispensável, mas depende dos dados de entrada, da máquina e do compilador. E custa tempo. Não dá para implementar seis alternativas para escolher uma.

Como decidir sem esperar 14 minutos?

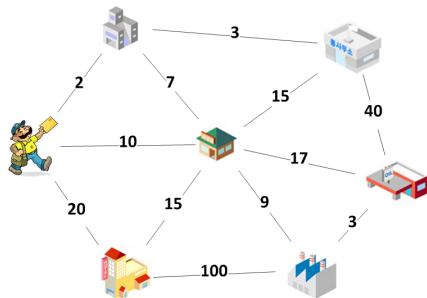
Análise empírica

Implementar, rodar, cronometrar. Indispensável, mas depende dos dados de entrada, da máquina e do compilador. E custa tempo. Não dá para implementar seis alternativas para escolher uma.

Análise de algoritmos

Estuda desempenho no papel, antes de existir código. Responde quanto tempo o algoritmo consome para uma entrada de tamanho n , sem depender da máquina.

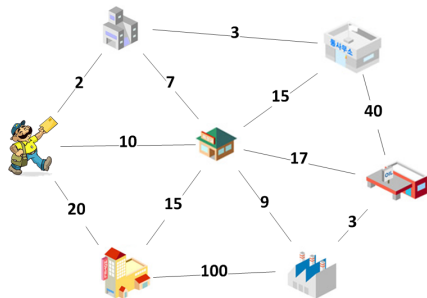
O problema do caixeiro viajante



Visitar n cidades, voltando à primeira. Qual rota minimiza a distância total?

A solução mais simples: testar todas as rotas e ficar com a menor.

O problema do caixeiro viajante



Visitar n cidades, voltando à primeira. Qual rota minimiza a distância total?

A solução mais simples: testar todas as rotas e ficar com a menor.

Com 4 cidades, fixando A como origem, são 6 rotas.

Em geral, $(n - 1)!$ rotas.

Fatorial contra polinomial, a 1 bilhão de adições por segundo

n	$(n-1)!$ rotas		n^5 alternativas	
	quantas	tempo	quantas	tempo
10	362.880	0,003 s	100.000	insignificante
15	87 bilhões	20 min	759.375	0,01 s
20	$1,2 \times 10^{17}$	73 anos	3.200.000	0,06 s
25	$6,2 \times 10^{23}$	470 milhões de anos	9.765.625	0,23 s

Fatorial contra polinomial, a 1 bilhão de adições por segundo

n	$(n - 1)!$ rotas		n^5 alternativas	
	quantas	tempo	quantas	tempo
10	362.880	0,003 s	100.000	insignificante
15	87 bilhões	20 min	759.375	0,01 s
20	$1,2 \times 10^{17}$	73 anos	3.200.000	0,06 s
25	$6,2 \times 10^{23}$	470 milhões de anos	9.765.625	0,23 s

Expoente 5 é alto, e o algoritmo seria considerado lento. Ainda assim resolve em décimos de segundo o que o outro não resolve em milhões de anos.

Fatorial contra polinomial, a 1 bilhão de adições por segundo

n	$(n-1)!$ rotas		n^5 alternativas	
	quantas	tempo	quantas	tempo
10	362.880	0,003 s	100.000	insignificante
15	87 bilhões	20 min	759.375	0,01 s
20	$1,2 \times 10^{17}$	73 anos	3.200.000	0,06 s
25	$6,2 \times 10^{23}$	470 milhões de anos	9.765.625	0,23 s

Expoente 5 é alto, e o algoritmo seria considerado lento. Ainda assim resolve em décimos de segundo o que o outro não resolve em milhões de anos.

Com uma máquina 1000 vezes mais rápida, os 73 anos caem para 26 dias. Some **uma** cidade e o ganho desaparece.

Modelo de custo: escolher o que contar

Contar instruções de máquina não serve, porque o número depende do compilador e do processador.

Sedgewick propõe fixar um **modelo de custo**: escolher a operação relevante para aquele algoritmo e contar só ela.

Ordenação	comparações e trocas
Busca	comparações com a chave
Union-Find	acessos ao array

O modelo é escolha do analista e precisa ser declarado junto com o resultado.

Quantas vezes contador++ executa?

```
for i := 0; i < n; i++ {           // (a)
    contador++
}
for i := 0; i < n; i++ {           // (b)
    for j := 0; j < n; j++ { contador++ }
}
for i := 0; i < n; i++ {           // (c)
    for j := i; j < n; j++ { contador++ }
}
```

Responda antes de continuar.

Laço interno que começa em i

(a) n (b) n^2

Laço interno que começa em i

$$(a) \ n \qquad (b) \ n^2$$

(c) o laço interno começa em i , então executa $n, n-1, \dots, 1$ vezes.

$$n + (n-1) + \dots + 1 = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}$$

Metade do trabalho de (b), e ainda assim os dois caem na mesma categoria.

Bubble Sort: quantas comparações no total?

```
func bubbleSort(v []int) int {  
    n := len(v)  
    comparacoes := 0  
    for i := 1; i < n; i++ {  
        for j := 0; j < n-i; j++ {  
            comparacoes++  
            if v[j] > v[j+1] {  
                v[j], v[j+1] = v[j+1], v[j]  
            }  
        }  
    }  
    return comparacoes  
}
```

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\text{Comparações} = \sum_{i=1}^{n-1} (n - i)$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1))\end{aligned}$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1)) \\ &= (n^2 - n) - \frac{n(n - 1)}{2}\end{aligned}$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1)) \\ &= (n^2 - n) - \frac{n(n - 1)}{2} \\ &= \frac{n^2}{2} - \frac{n}{2}\end{aligned}$$

Somando o laço interno

Para cada i de 1 a $n - 1$, o laço interno roda $n - i$ vezes.

$$\begin{aligned}\text{Comparações} &= \sum_{i=1}^{n-1} (n - i) \\ &= (n - 1) \cdot n - (1 + 2 + \cdots + (n - 1)) \\ &= (n^2 - n) - \frac{n(n - 1)}{2} \\ &= \frac{n^2}{2} - \frac{n}{2}\end{aligned}$$

A contagem não depende do conteúdo do vetor: este Bubble Sort sempre faz o mesmo número de comparações.

Termo dominante

n	$n/2$	$n^2/2$	$n^2/2 - n/2$
1024	512	524.288	523.776
32768	16.384	536.870.912	536.854.528

Na última linha, $n/2$ é três milésimos de 1% do total: existe, e é irrelevante.

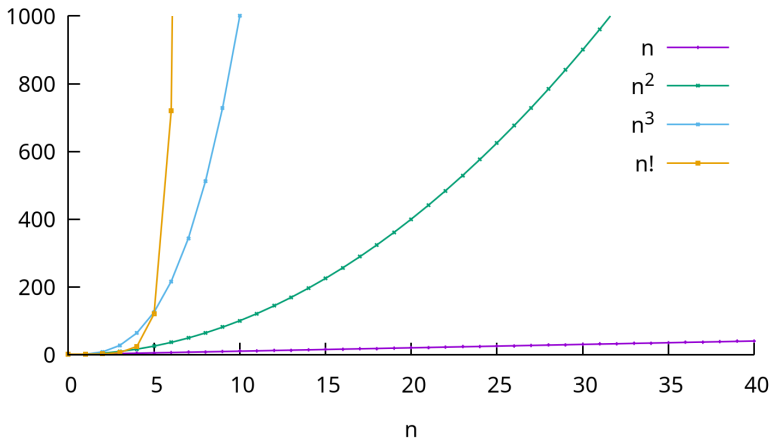
Duas simplificações

Descartar termos de ordem inferior: $n^2/2 - n/2$ vira $n^2/2$.

Descartar constantes multiplicativas: $n^2/2$ vira n^2 .

Válido **quando n é grande**. Problema pequeno, qualquer algoritmo resolve.

Crescimento comparado das classes



n , n^2 , n^3 e $n!$, para n até 40. O fatorial sai do gráfico antes de $n = 7$; o cúbico, em $n = 10$; o quadrático, perto de $n = 32$.

O , limite superior

$f(n) = O(g(n))$ quando existem $c > 0$ e $n_0 \geq 0$ tais que

$$f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0$$

A partir de certo n , f nunca ultrapassa g vezes uma constante.

Bubble Sort: $n^2/2 - n/2 = O(n^2)$. Também é $O(n^3)$, porque um limite frouxo continua sendo limite. Informe sempre o mais justo que conhecer.

Ω e Θ

Ω , limite inferior

$f(n) = \Omega(g(n))$: a partir de certo n , f é **pelo menos** g vezes uma constante.

Θ , limite justo

$f(n) = \Theta(g(n))$ quando $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.

Bubble Sort é $\Theta(n^2)$: o custo real, e não só um limite.

No dia a dia, escreve-se O quase sempre, mesmo quando Θ seria a afirmação mais forte. Saiba a diferença, use O .

Classes de crescimento

Classe	Função	Exemplo	Ao dobrar n
Constante	1	acesso a $v[i]$	não muda
Logarítmica	$\log n$	busca binária	soma constante
Linear	n	percorrer um slice	dobra
Linearítmica	$n \log n$	MergeSort	pouco mais que dobra
Quadrática	n^2	Bubble Sort	quadruplica
Cúbica	n^3	três laços	multiplica por 8
Exponencial	2^n	subconjuntos	inviável
Fatorial	$n!$	caixeiro viajante	inviável antes

A base do log não aparece: mudar de base só multiplica por uma constante. A última coluna é a ferramenta da próxima seção.

Melhor caso, pior caso, caso médio

Comparações do Bubble Sort não dependem do vetor. **Trocas** dependem:

- vetor já ordenado: 0 trocas (melhor caso);
- vetor decrescente: $n^2/2 - n/2$ trocas (pior caso).

QuickSort: $O(n^2)$ ou $O(n \log n)$?

Depende do caso. Pior caso: $O(n^2)$. Caso médio: $O(n \log n)$.

Dizer só $O(n^2)$ faz o algoritmo parecer pior do que ele costuma ser. Dizer só $O(n \log n)$ promete uma garantia que ele não dá.

Teste de duplicação

Como descobrir a classe sem ler o código

1. gere uma entrada aleatória de tamanho n ;
2. meça o tempo;
3. dobre n e repita;
4. observe para onde converge a razão entre tempos consecutivos.

Meça o binário compilado, não `go run`, que inclui o tempo de compilação. Descarte medições abaixo de um décimo de segundo, que são dominadas por ruído do sistema operacional.

Lei de potência e razão entre tempos

A maioria dos programas segue uma lei de potência: $T(n) = a \cdot n^b$, com a dependente da máquina e b igual à ordem de crescimento.

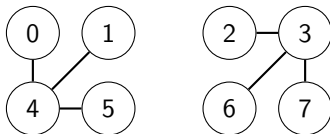
Dobrando a entrada:

$$\frac{T(2n)}{T(n)} = \frac{a \cdot (2n)^b}{a \cdot n^b} = 2^b$$

a desaparece na divisão. Só resta $b = \log_2(\text{razão})$.

Razão	2	4	8
Classe	linear	quadrática	cúbica

Componentes conectadas

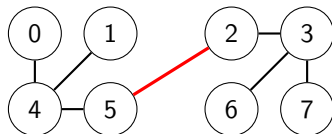


Componente conectada: conjunto maximal de objetos mutuamente conectados.

Aqui, $\{0, 1, 4, 5\}$ e $\{2, 3, 6, 7\}$.

O que muda se unirmos 2 e 5?

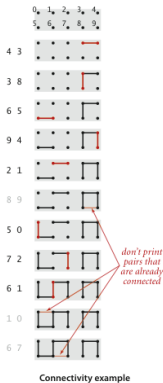
Componentes conectadas



Componente conectada: conjunto maximal de objetos mutuamente conectados.

Uma aresta nova, e as duas componentes viram $\{0, 1, 2, 3, 4, 5, 6, 7\}$. Nenhum outro par precisou ser informado: a relação é transitiva.

Retomando o problema



```
func NewUF(n int) *UF
func (uf *UF) Count() int
func (uf *UF) Find(p int) int
func (uf *UF) Connected(p, q int) bool
func (uf *UF) Union(p, q int)
```

Modelo de custo: acessos ao array (leitura ou escrita).

n é o número de objetos; M é o número de operações Union e Find ao longo da vida da estrutura.

Quick-Find: a estrutura

`id[i]` guarda a componente de `i`. Conectados $\iff$ mesmo valor em `id`.

```
func (uf *UF) Find(p int) int {  
    return uf.id[p]           // O(1)  
}  
func (uf *UF) Connected(p, q int) bool {  
    return uf.id[p] == uf.id[q] // O(1)  
}
```

E Union, quanto custa?

Quick-Find: o preço do Union

```
func (uf *UF) Union(p, q int) {  
    pID, qID := uf.id[p], uf.id[q]  
    if pID == qID { return }  
    for i := 0; i < uf.n; i++ {    // percorre TUDO  
        if uf.id[i] == pID {  
            uf.id[i] = qID  
        }  
    }  
    uf.count--  
}
```

$O(n)$ por união, sempre, mesmo unindo dois elementos isolados.

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		
union(4,3)	0	1	2	3	3	5	6	7	8	9	1	12

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		
union(4,3)	0	1	2	3	3	5	6	7	8	9	1	12
union(3,8)	0	1	2	8	8	5	6	7	8	9	2	12

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		
union(4,3)	0	1	2	3	3	5	6	7	8	9	1	12
union(3,8)	0	1	2	8	8	5	6	7	8	9	2	12
union(6,5)	0	1	2	8	8	5	5	7	8	9	1	12

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		
union(4,3)	0	1	2	3	3	5	6	7	8	9	1	12
union(3,8)	0	1	2	8	8	5	6	7	8	9	2	12
union(6,5)	0	1	2	8	8	5	5	7	8	9	1	12
union(9,4)	0	1	2	8	8	5	5	7	8	8	1	12

Quick-Find sobre tinyUF.txt

	0	1	2	3	4	5	6	7	8	9	escritas	leituras
início	0	1	2	3	4	5	6	7	8	9		
union(4,3)	0	1	2	3	3	5	6	7	8	9	1	12
union(3,8)	0	1	2	8	8	5	6	7	8	9	2	12
union(6,5)	0	1	2	8	8	5	5	7	8	9	1	12
union(9,4)	0	1	2	8	8	5	5	7	8	8	1	12

Poucas escritas, mas o laço **leu** as 10 posições em toda linha. Com n de 1 milhão, é 1 milhão de leituras por união.

M operações sobre n objetos custam $O(M \cdot n)$, quadrático quando M é proporcional a n .

Algoritmo quadrático não acompanha o hardware

Sedgewick: com 10^9 objetos e 10^9 uniões, o Quick-Find faria mais de 10^{18} acessos, ou mais de **30 anos** de processamento.

Algoritmo quadrático não acompanha o hardware

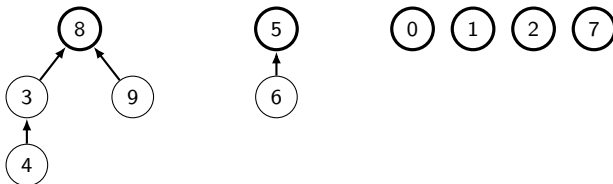
Sedgewick: com 10^9 objetos e 10^9 uniões, o Quick-Find faria mais de 10^{18} acessos, ou mais de **30 anos** de processamento.

Trocar por uma máquina 10 vezes mais rápida não resolve. Ela costuma ter 10 vezes mais memória, e será usada num problema 10 vezes maior, que o algoritmo quadrático demora **100 vezes mais** para processar.

Quick-Union: a estrutura

`parent[i]` guarda o **pai** de `i`, e não a componente. Quando `parent[i] == i`, `i` é **raiz**. Cada árvore é uma componente, e o slice representa uma floresta.

	0	1	2	3	4	5	6	7	8	9
parent	0	1	2	8	3	5	5	7	8	8



Depois de `union(4,3)`, `union(3,8)`, `union(6,5)` e `union(9,4)`. Raízes em traço grosso. **Invariante:** `p` e `q` estão conectados se, e somente se, têm a mesma raiz. `Find(4)` sobe 4, 3, 8.

Quick-Union: adiar o trabalho

Em vez de renomear a componente inteira, muda-se **um** valor. O custo migra para Find.

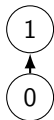
```
func (uf *UF) Find(p int) int {
    for p != uf.parent[p] {
        p = uf.parent[p]
    }
    return p
}

func (uf *UF) Union(p, q int) {
    rootP, rootQ := uf.Find(p), uf.Find(q)
    if rootP == rootQ { return }
    uf.parent[rootP] = rootQ
    uf.count--
}
```

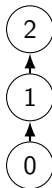
Union agora é uma atribuição. Qual o custo de Find?

Quick-Union: a cadeia cresce por baixo

union(0,1)



union(1,2)



union(2,3)



parent = [1 2 3 3] Find(0) sobe **3** arestas

A raiz **antiga** sempre passa a pender da nova. Com n uniões nessa ordem, a árvore vira uma lista de n nós e Find custa n : pior que o Quick-Find, e cada passo é um salto para posição arbitrária do slice, sem acesso contíguo à memória.

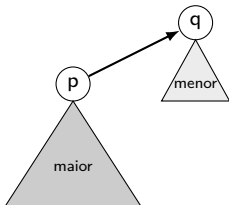
Union escolhe a raiz arbitrariamente

As árvores ficam fundas porque Union não usa **nenhuma** informação sobre as duas árvores que está unindo. Qualquer uma das raízes pode acabar embaixo.

Como você corrigiria isso?

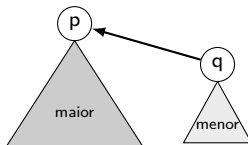
Sempre a menor sob a maior

Quick-Union: raiz arbitrária



a árvore **maior** desce
um nível inteiro

Weighted: sempre a mesma escolha



só a árvore **menor** desce

Guardar o tamanho de cada árvore e pendurar sempre a menor sob a raiz da maior, nunca o contrário.

Pendurar a menor sob a maior

```
if uf.size[rootP] < uf.size[rootQ] {
    uf.parent[rootP] = rootQ
    uf.size[rootQ] += uf.size[rootP]
} else {
    uf.parent[rootQ] = rootP
    uf.size[rootP] += uf.size[rootQ]
}
```

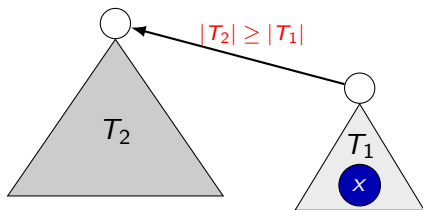
Três linhas a mais que o Quick-Union. Find é idêntico.

Proposição

No Weighted Quick-Union, a profundidade de qualquer nó é no máximo $\log_2 n$.

Com $n = 1.000.000$, isso são 20 níveis, contra o milhão de acessos que uma união do Quick-Find faria.

Por que a profundidade cresce tão pouco



A profundidade de x só aumenta quando a árvore que o contém (T_1) é pendurada sob outra (T_2), e isso só acontece quando T_1 é a menor das duas.

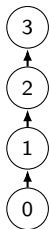
A árvore resultante tem $|T_1| + |T_2|$ elementos. Como T_1 é a menor, $|T_2| \geq |T_1|$, e portanto

$$|T_1| + |T_2| \geq |T_1| + |T_1| = 2|T_1|$$

A árvore de x **pelo menos dobrou**. Como nenhuma árvore passa de n elementos, dobrar acontece no máximo $\log_2 n$ vezes.

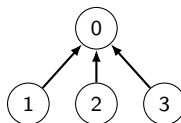
A mesma sequência nas duas implementações

Quick-Union



altura **3**

Weighted Quick-Union



altura **1**

`union(0,1), union(1,2), union(2,3)`

Mesmos pares, mesma ordem, mesma resposta a Connected. O que mudou foi qual raiz pende de qual.

Custo de M operações sobre n objetos

Implementação	Custo	Ordem
Quick-Find	$M \cdot n$	quadrática
Quick-Union	$M \cdot n$	quadrática (pior caso)
Weighted Quick-Union	$n + M \log n$	linearítmica

Você mediria essas três com o teste de duplicação.
Que razão esperaria ver?

Tempos medidos, três implementações

n	Quick-Find	razão	Quick-Union	razão	Weighted
20.000	0,31 s		0,30 s		0,01 s
40.000	1,23 s	4,0	1,48 s	4,9	0,02 s
80.000	4,96 s	4,0	6,95 s	4,7	0,06 s
160.000	19,63 s	4,0	35,28 s	5,1	0,12 s

Razão 4 confirma quadrático ($b = \log_2 4 = 2$), exatamente como a contagem de acessos previa. No Quick-Union a razão passa de 4, porque o custo de cada operação também cresce com n .

Go 1.26, Intel Core i5-1345U, entradas com n objetos e $2n$ pares aleatórios. Os tempos do Weighted são pequenos demais para render razão confiável nesses tamanhos.

Extrapolar cinco medições para uma execução de 14 minutos

De 160.000 para 1.000.000 de objetos, n é multiplicado por 6,25.
Sendo quadrático, o tempo deve ser multiplicado por $6,25^2 = 39$:

$$19,63 \text{ s} \times 39 = 766 \text{ s} = 12\text{min}46\text{s}$$

Extrapolar cinco medições para uma execução de 14 minutos

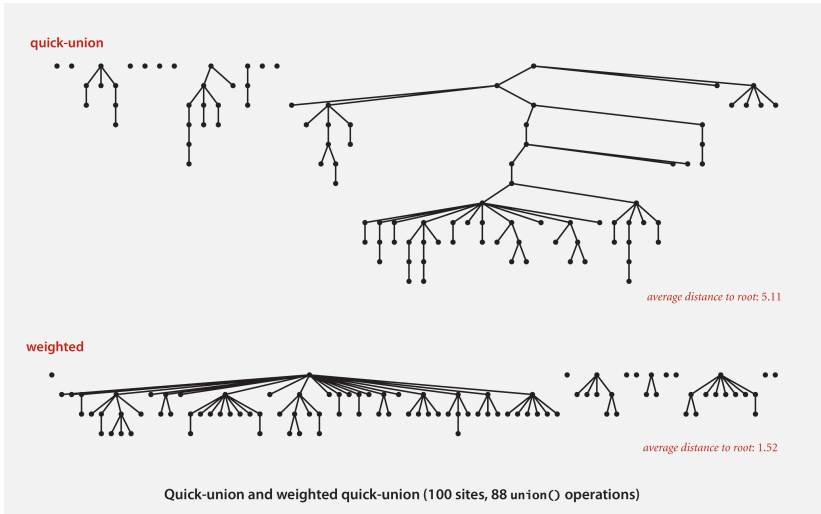
De 160.000 para 1.000.000 de objetos, n é multiplicado por 6,25. Sendo quadrático, o tempo deve ser multiplicado por $6,25^2 = 39$:

$$19,63 \text{ s} \times 39 = 766 \text{ s} = 12\text{min}46\text{s}$$

largeUF.txt (10^6 objetos)	previsto	medido
Quick-Find	766 s	829 s
Weighted Quick-Union		1,07 s

Erro de 8% numa extrapolação seis vezes além do maior tamanho medido. São os mesmos 14 minutos do início da aula, agora previstos sem executá-los.

A forma da árvore explica o número



100 sites, 88 uniões. Quick-Union produz árvores altas (distância média à raiz: 5,11); Weighted mantém tudo raso (1,52). Figura: Sedgwick & Wayne, *Algorithms*, 4th ed., slides do capítulo 1.5.

Qual implementação usar

- Dezenas ou centenas de objetos: qualquer uma responde em microssegundos, e a escolha pode seguir a simplicidade do código;
- Milhares de objetos com uso intenso de `Union`: a ordem de crescimento domina, e `Weighted Quick-Union` é a escolha.

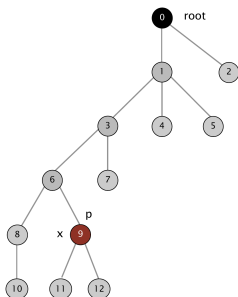
O ganho não vem de código mais rápido. As três têm o mesmo tipo de laço, sobre o mesmo tipo de slice. Vem de escolher a estrutura de dados que evita trabalho.

Resumo

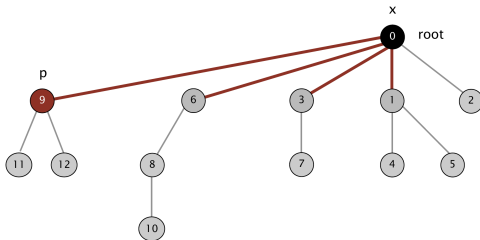
- Medição depende de máquina, dados e implementação. Análise assintótica não depende de nenhum dos três;
- Analisar começa por declarar o modelo de custo;
- Termos de ordem inferior e constantes somem quando n cresce. Para n pequeno, eles dominam;
- O é limite superior, Ω inferior, Θ os dois. Diga sempre qual caso;
- Razão ao dobrar n : 2 linear, 4 quadrática, 8 cúbica;
- Quick-Find: Find $O(1)$, Union $O(n)$. Quick-Union inverte, e degenera em árvore torta;
- Pendurar a menor sob a maior limita a profundidade a $\log_2 n$, e separa 14 minutos de 1 segundo em `largeUF.txt`.

Compressão de caminho

Ao subir até a raiz, $\text{Find}(p)$ passa por **todos** os nós do caminho e descobre a raiz de cada um. Apontar cada um deles direto para a raiz custa pouco e deixa a árvore plana.



antes



depois de $\text{Find}(9)$

$\text{Find}(9)$ examina 9, 6, 3, 1 e 0. Os quatro primeiros passam a apontar direto para a raiz, e as consultas seguintes a qualquer um deles ficam mais baratas.

Quão barato o Union-Find fica

Com compressão de caminho, o custo de M operações sobre n objetos cai para $n + M \lg^* n$, onde $\lg^* n$ é o logaritmo iterado: quantas vezes é preciso aplicar $\log_2$ a n até chegar a 1.

Ele vale 5 para $n = 2^{65536}$, número maior que a quantidade de átomos no universo observável. Para qualquer entrada concebível, $\lg^* n \leq 5$.

Leitura, não vista em aula

A seção 15 do texto trata de compressão de caminho, do limite de Hopcroft-Ulman-Tarjan e das aplicações do Union-Find. Não cai na prova, e é ponto de partida para o desafio da lista.

Bibliografia

- SEDGEWICK, R.; WAYNE, K. **Algorithms**, 4th ed., seções 1.4 e 1.5.
- <https://algs4.cs.princeton.edu/14analysis/>
- <https://algs4.cs.princeton.edu/15uf/>
- BHARGAVA, A. Y. **Entendendo Algoritmos**, cap. 1 e 4.